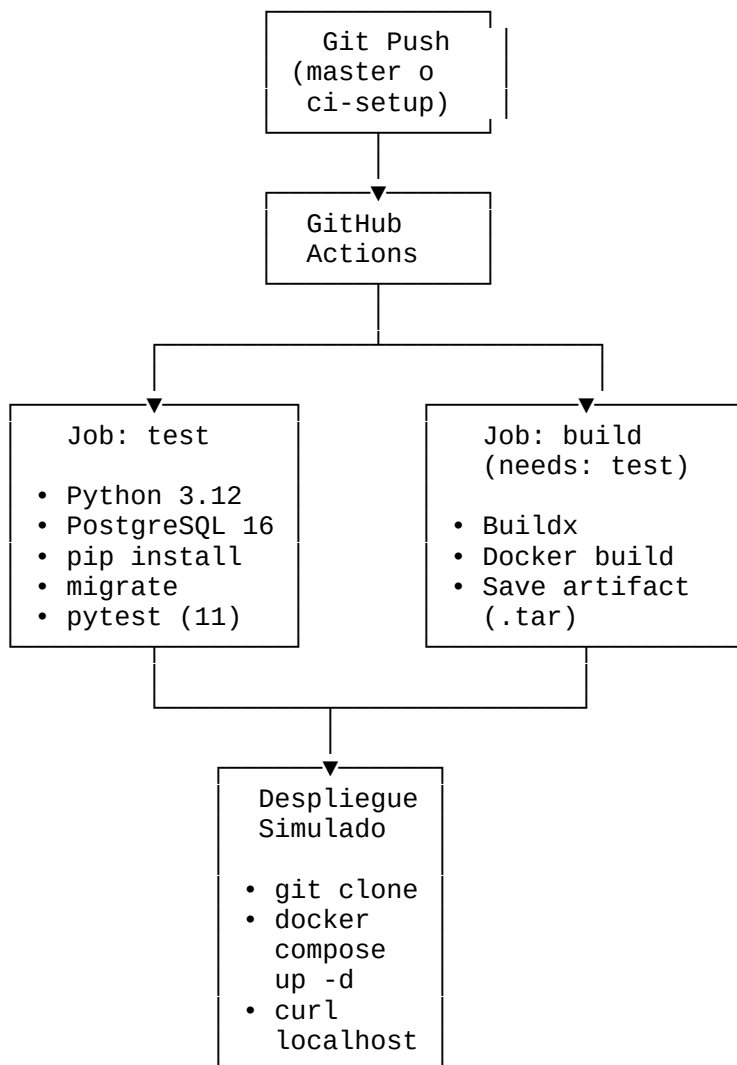


Flujo CI/CD para Django – GCS Store

1. Diagrama del Pipeline

...



...

2. Explicación del archivo ``.github/workflows/ci.yml``

Estructura

El pipeline consta de **2 jobs** que se ejecutan secuencialmente:

Job 1: `test`

```yaml

```
test:
 runs-on: ubuntu-latest
 services:
 postgres:
 image: postgres:16
```

...

```

- **Propósito**: Ejecutar los tests unitarios de Django.
- **Base de datos**: Levanta un contenedor temporal de PostgreSQL 16 con healthcheck.
- **Pasos**:
 1. `actions/checkout@v4` – Clona el repositorio.
 2. `actions/setup-python@v5` – Configura Python 3.12.
 3. `pip install -r requirements.txt` – Instala Django, psycopg2-binary y dependencias.
 4. `python manage.py migrate` – Ejecuta migraciones contra PostgreSQL.
 5. `python manage.py test` – Corre 11 tests (modelos Producto, Cliente, Pedido, DetallePedido y vistas).
- **Variables de entorno**: `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD` se pasan para conectar con PostgreSQL.

Job 2: `build`

```yaml
build:
  needs: test
  runs-on: ubuntu-latest
```

- **Propósito**: Construir la imagen Docker y guardarla como artefacto.
- **Dependencia**: Solo se ejecuta si `test` finaliza con éxito (`needs: test`).
- **Pasos**:
 1. `docker/setup-buildx-action@v3` – Configura Docker Buildx.
 2. `docker/build-push-action@v6` – Construye la imagen con `push: false` y la exporta a `/tmp/gcs-app.tar`.
 3. `actions/upload-artifact@v4` – Sube el `.tar` como artefacto descargable por 7 días.

Simulación del despliegue (local)

```powershell
git clone --branch ci-setup <repo> deploy_demo
cd deploy_demo
docker compose up -d --build
```

- **Clona** el proyecto desde GitHub.
- **Construye y levanta** los contenedores (Django + PostgreSQL).
- **Valida** abriendo `http://localhost:8000/` en el navegador.

```

### ## 3. Recomendaciones Finales

| Aspecto                     | Recomendación                                                                                            |
|-----------------------------|----------------------------------------------------------------------------------------------------------|
| <b>**Seguridad**</b>        | Usar GitHub Secrets para contraseñas y tokens, no hardcodear.                                            |
| <b>**Base de datos**</b>    | En producción, no usar PostgreSQL como servicio interno de CI; conectar a una BD externa.                |
| <b>**Artefacto Docker**</b> | Si se requiere distribución, pushear a DockerHub o GitHub Container Registry en lugar de subir artifact. |
| <b>**Tests**</b>            | Agregar tests de integración con Selenium/Playwright para cubrir frontend.                               |
| <b>**Ramas**</b>            | Proteger `master` con branch rules: requerir CI pass antes de mergear.                                   |
| <b>**Entorno**</b>          | Usar `.env` para configuraciones sensibles; no commiteados.                                              |
| <b>**Versiones**</b>        | Congelar versiones en `requirements.txt` para builds reproducibles.                                      |

#### ## 4. Comandos de Verificación

```
```bash
```

```
# Tests locales
```

```
python manage.py test
```

```
# Construir y correr con Docker
```

```
docker compose up -d --build
```

```
# Ver contenedores
```

```
docker compose ps
```

```
# Logs
```

```
docker compose logs web -f
```

```
# Detener
```

```
docker compose down -v
```

```
```
```

```

```

```
> **Asignatura**: Gestión de Configuración del Software
```

```
> **Repositorio**: https://github.com/IngAdancito/S8-TRABAJO-PR-CTICO-EXPERIMENTAL_4
```

```
> **Rama**: `ci-setup`
```